

Research IQ

AI-Powered Research Paper Assistant

Hybrid RAG System using FAISS + BM25 + Cross-Encoder Reranking + LLM

Prepared by: Kishan Rabari

1. Problem Statement

Researchers and students spend a significant amount of time manually searching through and reading research papers to find relevant information. Traditional keyword-based search engines often fail to understand the context or intent behind a query, returning results that are either too broad or miss semantically relevant papers altogether.

There is a need for an intelligent system that can understand natural language questions, retrieve the most contextually relevant research content, and generate accurate, source-backed answers — reducing the time and effort required for literature review and research discovery.

Research IQ solves this by combining semantic (vector) search with keyword-based (BM25) search, followed by cross-encoder reranking, and passing the most relevant context to a large language model to generate a grounded, natural-language answer.

2. Objectives

- Automatically fetch research papers from arXiv based on a topic query.
- Store paper metadata (title, authors, abstract, date, link) in a structured database.
- Convert paper content into searchable chunks and semantic vector embeddings.
- Combine keyword search (BM25) and semantic search (FAISS) for higher retrieval accuracy.
- Rerank retrieved results using a Cross-Encoder for better relevance.
- Generate accurate, context-grounded answers using a local LLM (Ollama).
- Provide a simple web-based frontend for users to search and view results.

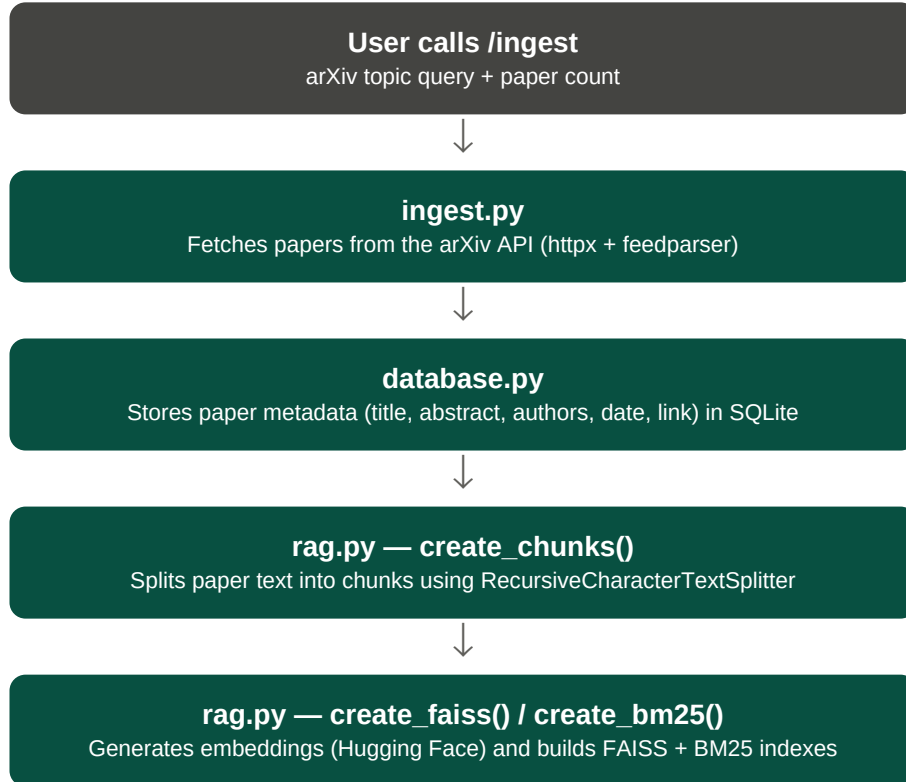
3. Technology Stack

Layer	Technology	Purpose
Backend Framework	FastAPI	Exposes REST API endpoints
Data Source	arXiv API (httpx + feedparser)	Fetch research papers
Metadata Storage	SQLite (sqlite3)	Stores paper metadata
Chunking	LangChain RecursiveCharacterTextSplitter	Splits text into meaningful chunks
Embeddings	Hugging Face sentence-transformers (all-MiniLM-L6-v2)	Converts text into vector embeddings
Vector Search	FAISS (IndexFlatIP)	Fast semantic similarity search
Keyword Search	rank-bm25 (BM25Okapi)	Classic keyword-based search
Result Fusion	Reciprocal Rank Fusion (RRF)	Merges BM25 + vector search results
Reranking	Cross-Encoder (ms-marco-MiniLM-L-6-v2)	Reorders results by true relevance
Answer Generation	Ollama (llama3.2)	Generates the final natural-language answer
Frontend	HTML, CSS, JavaScript	User interface for search and results

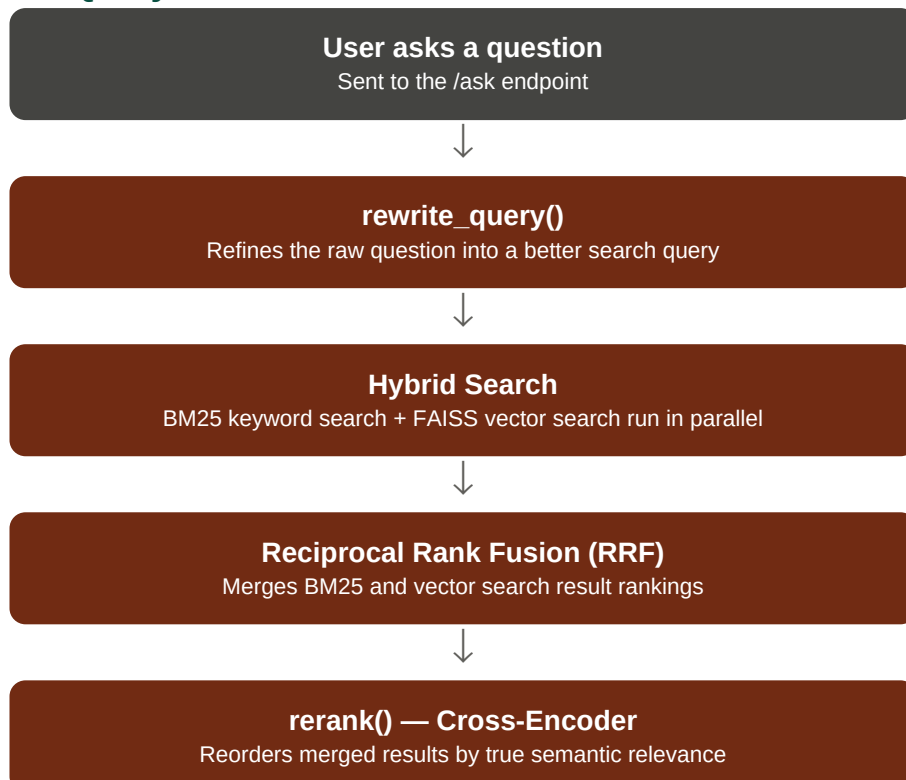
4. System Architecture & Flow

The system works in two major phases: **(A) Data Ingestion** — where papers are fetched, stored, chunked, embedded and indexed; and **(B) Query & Answer Generation** — where a user's question is converted into a grounded answer using hybrid retrieval and an LLM.

4.1 Phase A — Data Ingestion Flow



4.2 Phase B — Query & Answer Generation Flow





ask_ollama()

Sends top chunks as context to the LLM, which generates the final answer

5. File-by-File Code Explanation

database.py — Storage Layer

- Creates and manages the SQLite database (data/papers.db).
- `create_table()` — creates the papers table if it does not exist.
- `save_papers()` — inserts fetched paper metadata into the database.
- `get_papers()` — retrieves all stored papers for chunking/indexing or listing.

ingest.py — Data Fetching

- Connects to the arXiv API using `httplib` and parses the Atom feed using `feedparser`.
- `fetch_papers(query, start, max_results)` fetches papers in batches based on a topic query.
- Extracted data (title, abstract, authors, date, link) is passed to `database.py` to be stored.

rag.py (Part 1) — Chunking, Embedding, Indexing

- `create_chunks()` — splits each paper's title + abstract into overlapping chunks using LangChain's `RecursiveCharacterTextSplitter` (`chunk_size=1000` characters, `overlap=150` characters). This preserves sentence and paragraph boundaries instead of cutting text mid-sentence.
- `load_embedding_model()` — loads the Hugging Face sentence-transformers model (all-MiniLM-L6-v2).
- `create_faiss()` — encodes all chunks into normalized embeddings and stores them in a FAISS `IndexFlatIP` (exact inner-product search).
- `create_bm25()` — builds a BM25Okapi keyword-search index over the same chunks.
- Data is persisted to disk as `chunks.pkl`, `bm25.pkl` (Python pickle) and `faiss.index` (FAISS binary format).

rag.py (Part 2) — Hybrid Search, Rerank, LLM Answer

- `rewrite_query()` — sends the user's question to the LLM to produce a cleaner search query while preserving key entities and keywords.
- `bm25_search()` — scores chunks using BM25 keyword matching and returns the top-k candidates.
- `vector_search()` — embeds the query and searches the FAISS index for the top-k semantically similar chunks.
- `reciprocal_rank_fusion()` — merges the BM25 and vector search rankings into a single combined ranking using the RRF formula.
- `rerank()` — uses a Cross-Encoder model to directly score (query, chunk) pairs and picks the final top-k most relevant chunks.
- `ask_ollama()` — builds a prompt containing the retrieved context and the user's question, sends it to a locally running Ollama LLM (llama3.2), and returns the generated answer.

main.py — API Layer (FastAPI)

- Exposes the backend as a REST API using FastAPI, with CORS middleware enabled so the frontend (running on a different port) can call it.
- `POST /ingest` — triggers fetching new papers and rebuilding the indexes.
- `GET /papers` — returns the list of stored papers.
- `GET /ask` — accepts a user question and returns the generated answer with source citations.

Frontend (index.html, script.js, style.css)

- Provides a simple browser-based interface: a search box to ask questions, a table to view ingested papers, and a results panel to display the generated answer along with its sources.
- Communicates with the backend purely via `fetch()` calls to the FastAPI endpoints.

6. What Each File in data/ Stores

File	Contents	Format / Why
faiss.index	Vector embeddings of all chunks, organized for fast similarity search	FAISS's own binary index format (faiss.write_index) — required by the FAISS C++ engine
bm25.pkl	The full trained BM25Okapi object (term/document frequencies used for keyword scoring)	Python pickle — saves the Python object as-is
chunks.pkl	The list of text chunks with their metadata (title, link, authors, date)	Python pickle — simple list/dict structure
papers.db	Structured paper metadata (title, abstract, authors, date, link)	SQLite database — supports querying, filtering, and relational storage

7. Key Design Decisions

Why IndexFlatIP for FAISS?

IndexFlatIP performs exact (brute-force) similarity search using the inner product. Combined with normalized embeddings, this is mathematically equivalent to cosine similarity. Since the dataset is in the hundreds/low thousands of chunks (not millions), an exact search is fast enough and avoids the accuracy loss of approximate index types like IVF or HNSW.

Why RecursiveCharacterTextSplitter for chunking?

Instead of naively cutting text every N words (which can split a sentence in half and hurt embedding quality), the recursive splitter tries paragraph, then sentence, then word boundaries before falling back to a hard cut — keeping chunks semantically coherent. A 150-character overlap between chunks also preserves context across chunk boundaries.

Why Hybrid Search (BM25 + FAISS) instead of vector search alone?

Vector search is great at semantic/contextual matching but can miss exact keyword matches (e.g. rare technical terms, model names). BM25 is great at exact keyword matching but misses semantic similarity. Combining both via Reciprocal Rank Fusion captures the strengths of each.

Why rerank with a Cross-Encoder?

Bi-encoder embeddings (used for FAISS) compare query and document independently, which is fast but less precise. A Cross-Encoder scores the query and each candidate chunk together, producing a much more accurate relevance ranking — used here as a final precision pass on a small candidate set (fast enough since only the top-k candidates are reranked).

Why does the system sometimes say “I could not find enough information”?

This is a deliberate grounding safeguard in the LLM prompt: if the retrieved context does not actually contain information relevant to the question, the model is instructed to say so rather than fabricate an answer. This prevents hallucination and keeps answers trustworthy.

Why allow_origins=["*"] in CORS middleware?

This allows the frontend (served from a different port/origin than the FastAPI backend) to call the API during development without CORS errors. In a production deployment this would be restricted to the specific frontend domain.

8. Limitations & Future Improvements

- Currently only paper title + abstract are ingested; extending to full paper PDF text would require more robust chunking and larger-scale indexing (e.g. IndexIVFFlat or HNSW for speed).
- Query rewriting and answer generation depend on a locally running Ollama instance — this could be made configurable to use a hosted LLM API as a fallback.
- No authentication/authorization on the API endpoints — would be needed for a multi-user deployment.
- CORS is fully open (allow_origins="*") — should be restricted to specific domains in production.
- Retrieval quality could be further improved with query expansion, metadata filtering (date/author), and citation-aware chunking.

9. Conclusion

Research IQ demonstrates a complete, production-style Retrieval-Augmented Generation pipeline: automated data collection, structured storage, semantic + keyword hybrid retrieval, cross-encoder reranking, and grounded LLM-based answer generation — all wrapped in a simple, usable web interface. The architecture reflects real-world RAG system design choices and is built to be extended (full-text ingestion, larger-scale vector indexing, and production security hardening).